

Para o problema de **buscar o maior elemento em um vetor**, podemos estabelecer a função de complexidade seguindo o mesmo modelo da aula 02.

- **Comparação:** custo 1
- **Atribuição:** custo 2
- **Operação matemática:** custo 3

Vou considerar o algoritmo clássico:

```
int maior(int V[], int n) {  
    int maior = V[0];  
  
    for (int i = 1; i < n; i++) {  
        if (V[i] > maior) {  
            maior = V[i];  
        }  
    }  
}
```

```
    return maior;  
}
```

1. Analisando cada instrução

Inicialização do maior

`maior = V[0];`

É uma atribuição:

$$C = 2$$

Inicialização do contador

`i = 1`

Outra atribuição:

$$C = 2$$

Condição do for

`i < n`

O teste é realizado **n vezes**:

- uma vez para cada elemento processado;
- mais uma vez quando `i == n` para encerrar o laço.

Portanto:

$$C = n$$

Como é comparação e custa 1:

$$C = n$$

Comparação dos elementos

`V[i] > maior`

Essa comparação ocorre para os elementos:

`V[1], V[2], ..., V[n-1]`

Logo:

$$n - 1$$

comparações.

Como cada comparação custa 1:

$$C = n - 1$$

Atualização do maior

`maior = V[i];`

Essa atribuição ocorre somente quando encontramos um elemento maior.
Se isso acontecer k vezes:

$$C = 2k$$

No **pior caso**, por exemplo:

1 2 3 4 5 6

↑ ↑ ↑ ↑ ↑

praticamente todos os elementos são maiores que o anterior.

Nesse caso:

$$k = n - 1$$

e:

$$C = 2(n - 1)$$

Incremento do contador

$i++$

Temos uma operação matemática para cada repetição, exceto na última:

$$n - 1$$

operações.

Como cada operação matemática custa 3:

$$C = 3(n - 1)$$

2. Montando a função de complexidade

Agora podemos somar todos os custos:

$$T(n) = 2 + 2 + n + (n - 1) + 2k + 3(n - 1)$$

Organizando:

$$T(n) = 5n + 2k$$

Essa é uma forma de representar a função considerando que o número de vezes que o maior é atualizado depende dos dados do vetor.

3. Pior caso

No pior caso:

$$k = n - 1$$

Substituindo:

$$T(n) = 5n + 2(n - 1)$$

$$T(n) = 5n + 2n - 2$$

Portanto:

$$\boxed{T(n) = 7n - 2}$$

Essa é a **função de custo no pior caso**, considerando exatamente os custos estabelecidos no exercício.

4. E a complexidade assintótica?

Agora vem a parte mais importante.

Temos:

$$T(n) = 7n - 2$$

Na análise assintótica, ignoramos:

- constantes multiplicativas;
- termos constantes.

Assim:

$$7n - 2 \rightarrow O(n)$$

Portanto:

$$\text{Complexidade} = O(n)$$

É uma **complexidade linear**.

5. Por que é $O(n)$?

Porque precisamos verificar todos os elementos do vetor.

Por exemplo:

$V = [15, 8, 32, 7, 45, 21]$

O algoritmo faz:

15 → maior

8 → não é maior

32 → novo maior

7 → não é maior

45 → novo maior

21 → não é maior

Precisamos passar por:

15 → 8 → 32 → 7 → 45 → 21

↑ ↑
início fim

Não podemos simplesmente parar quando encontramos 45, porque ainda precisaríamos saber se existe um elemento maior depois dele.

6. Resumo da análise

Instrução	Quantidade	Custo	Custo total
maior = V[0]	1	2	2
i = 1	1	2	2
i < n	n	1	n
V[i] > maior	n - 1	1	n - 1
maior = V[i]	k	2	2k
i++	n - 1	3	3(n - 1)
Total			5n + 2k

No pior caso:

$$T(n) = 7n - 2$$

E a complexidade:

$$O(n)$$

A função de complexidade do algoritmo de busca do maior elemento em um vetor é linear. Considerando comparação com custo 1, atribuição com custo 2 e operação matemática com custo 3, temos $T(n) = 5n + 2k$, onde k representa o número de atualizações do maior elemento. No pior caso, $k = n - 1$, resultando em $T(n) = 7n - 2$. Portanto, desprezando constantes e termos de menor ordem, a complexidade assintótica do algoritmo é $O(n)$.